

ASSESSMENT I - APRIL 2, 2025

Started on	Wednesday, 2 April 2025, 11:00
State	Finished
Completed on	Wednesday, 2 April 2025, 11:24
Time taken	
Grade	100.00 out of 100.00

Question 1

Assume that there are **4 processes**, P1 through P4, and **3 types of resources**: A, B and C.

At time T0, let consider the following snapshot of the system:

Process	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P1	0	1	0	7	5	5	2	3	0
P2	3	0	2	3	2	2			
P3	3	0	2	9	0	2			
P4	2	1	1	2	2	2			

Currently the system in a safe state.

What is the execution order of the processes so that the system remains in a safe state?

- ☐ P1 P2 P3 P4
- ☐ P3 P1 P2 P4
- ☐ P1 P3 P2 P4
- ☐ P4 P3 P1 P2
- ☐ P2 P1 P3 P4
- ☐ P3 P1 P4 P2
- ☐ P4 P1 P2 P3
- ☒ P2 P4 P3 P1 ✓

Solution

Available resource (A B C) = (2 3 0)

Process	Need		
	A	B	C
P1	7	4	5
P2	0	2	0
P3	6	0	0
P4	0	1	1

P2: (2 3 0) + (3 0 2) = (5 3 2)

P4: (2 1 1) + (5 3 2) = (7 4 3)

P3: (3 0 2) + (7 4 3) = (10 4 5)

P1: (0 1 0) + (10 4 5) = (10 5 5)

Question 2

Process **aging** is:

- ☐ Giving a process a longer quantum as it gets older.
- ☐ Computing the next CPU burst time via a weighted exponential average of previous bursts.
- ☒ Boosting a process' priority temporarily to get it scheduled to run. ✓
- ☐ The measurement of elapsed CPU time during a process' execution.

[题目 2] **Process aging is:**

翻译：进程老化是指：

- A. Giving a process a longer quantum as it gets older. (随着进程存在时间变长，给它更长的时间片。)
- B. Computing the next CPU burst time via a weighted exponential average of previous bursts. (通过之前 CPU 突发时间的加权指数平均来计算下一个 CPU 突发时间。)
- C. Boosting a process' priority temporarily to get it scheduled to run. (临时提升一个进程的优先级以便使其能被调度运行。)
- D. The measurement of elapsed CPU time during a process' execution. (测量进程执行期间已过去的 CPU 时间。)

答案：C

解释：进程老化是一种应对饥饿现象的策略。在基于优先级的调度系统中，若低优先级进程长时间等待，可能永远无法执行，通过进程老化，临时提升这些长时间等待进程的优先级，使它们有机会被调度运行，C 选项符合。A 选项描述的是对时间片的调整，并非老化概念；B 选项是对 CPU 突发时间的计算方式，和老化无关；D 选项只是对进程执行时 CPU 时间的测量，也不是老化的定义。所以答案是 C。

Question 3

_____ is a **non-preemptive scheduling algorithm** that handles jobs based on the length of their CPU burst time.

- ☒ Shortest job first (SJF) algorithm ✓
- ☐ Round Robin (RR) algorithm
- ☐ Priority algorithm
- ☐ First-Come First-Served (FCFS) algorithm
- ☐ none of the mentioned

[题目 3] ____ is a non - preemptive scheduling algorithm that handles jobs based on the length of their CPU burst time.

翻译：____是一种非抢占式调度算法，它根据作业的 CPU 突发时间长度来处理作业。

A. Shortest job first (SJF) algorithm (最短作业优先 (SJF) 算法)

最短作业优先算法可以分为非抢占式和抢占式。非抢占式的 SJF 算法会根据作业的 CPU 突发时间 (即预计运行时间) 来调度作业，优先调度 CPU 突发时间最短的作业，且一旦一个作业开始运行，直到它完成或者主动放弃 CPU 才会调度其他作业，符合题意。

B. Round Robin (RR) algorithm (循环调度 (RR) 算法)

循环调度算法是一种抢占式调度算法，它为每个进程分配固定的时间片，时间片用完后会剥夺该进程的 CPU 使用权，不符合非抢占式以及根据 CPU 突发时间调度的特点，所以 B 选项错误。

C. Priority algorithm（优先级算法）

优先级算法根据进程的优先级来调度，虽然有些优先级算法可能会考虑 CPU 突发时间等因素来确定优先级，但它并不单纯基于 CPU 突发时间，且有抢占式和非抢占式多种情况，不符合题意，C 选项错误。

D. First - Come First - Served (FCFS) algorithm（先来先服务（FCFS）算法）

先来先服务算法按照进程到达的先后顺序进行调度，而不是基于 CPU 突发时间，所以 D 选项错误。

答案：A

解释：在上述选项中，只有最短作业优先（SJF）算法既可以是非抢占式的，又基于作业的 CPU 突发时间长度来处理作业。其他选项的调度算法特点均不符合题目描述。所以答案是 A 选项，即最短作业优先（SJF）算法。

Question 4

One example of a hardware solution to the **critical section problem** is:

- ☒ Test and Set. ✓
- ☐ Peterson's Algorithm.
- ☐ Compare and Pray.
- ☐ Compare and Shop.
- ☐ Banker's Algorithm.

[题目 4] **One example of a hardware solution to the critical section problem is:**

翻译：临界区问题的一个硬件解决方案示例是：

A. Test and Set（测试与设置）

Test and Set（通常缩写为TS或TSL）是一种通过硬件指令实现的解决临界区问题的方法。它使用一个特殊的硬件指令来原子性地测试和设置一个标志位，以此来控制对临界区的访问，是硬件解决方案，A选项正确。

B. Peterson's Algorithm（彼得森算法）

彼得森算法是一种基于软件的解决临界区问题的方法，通过软件逻辑来协调进程对临界区的访问，并非硬件解决方案，B选项错误。

C. Compare and Pray（不存在这样一种用于解决临界区问题的标准算法，此为干扰项）

D. Compare and Shop（不存在这样一种用于解决临界区问题的标准算法，此为干扰项）

E. Banker's Algorithm（银行家算法）

银行家算法用于避免死锁，主要解决资源分配与死锁预防问题，与临界区问题的解决方案无关，E选项错误。

答案：A

解释：在上述选项中，只有Test and Set是通过硬件指令来实现对临界区访问控制的解决方案。彼得森算法是软件层面的方法，银行家算法用于死锁避免，而C、D选项为无意义干扰项。所以答案是A，即Test and Set是临界区问题的硬件解决方案示例。

Question 5

Consider a **Real-Time System** in which there are three processes. Their period and execution time are as follows:

Processes	Execution time, e	Period, p
P1	35	100
P2	10	50
P3	30	150

Answer:

The total utilization of processor is 75 %.

$$\text{utilization} = \frac{35}{100} + \frac{10}{50} + \frac{30}{150} = 0.35 + 0.2 + 0.2 = 0.75 = 75\%$$

Question 6

Among CPU scheduling policies, **First Come First Serve (FCFS)** algorithm is attractive because:

- ☒ it is simple to implement. ✓
- ☐ it is fair to all processes.
- ☐ it minimizes the average turnaround time in the system.
- ☐ it minimizes the average waiting time in the system.

[题目 6] Among CPU scheduling policies, First Come First Serve (FCFS) algorithm is attractive because:

翻译：在 CPU 调度策略中，先来先服务（FCFS）算法具有吸引力是因为：

A. it is simple to implement. （它实现起来很简单。）

B. it is fair to all processes. （它对所有进程都公平。）

C. it minimizes the average turnaround time in the system. （它使系统中的平均周转时间最小化。）

D. it minimizes the average waiting time in the system. （它使系统中的平均等待时间最小化。）

答案：A

解释：先来先服务算法实现起来非常简单，只需要按照进程到达的先后顺序进行调度即可，这是它的一个显著优点，A 选项正确。实际上，FCFS 算法并不一定对所有进程公平，比如长作业可能会使短作业等待很长时间，B 选项错误；FCFS 算法一般不会使平均周转时间和平均等待时间最小化，因为长作业在前会导致后续短作业等待时间过长，从而增加平均周转时间和平均等待时间，C、D 选项错误。

Question 7

Which of the following conditions is enforced by using **wait()** and **signal()** operations on semaphores?

- ☐ Non preemption.
- ☐ Aging.
- ☐ Resource holding.
- ☒ Mutual exclusion. ✓
- ☐ Starvation.

[题目 7] Which of the following conditions is enforced by using wait() and signal() operations on semaphores?

翻译：通过对信号量使用 wait() 和 signal() 操作来实施以下哪种条件？

- A. Non preemption (非抢占)
- B. Aging (老化)
- C. Resource holding (资源持有)
- D. Mutual exclusion (互斥)
- E. Starvation (饥饿)

答案：C

解释：信号量的wait()操作可用于获取资源，当信号量值大于0时，获取信号量（值减1）意味着进程持有了资源；signal()操作则用于释放资源（信号量值加1）。通过这两个操作可以有效地管理进程对资源的持有与释放。虽然互斥也是信号量常见用途，但这里答案强调资源持有。非抢占与调度机制相关，并非信号量wait()和signal()操作直接达成，A选项错误；老化主要是解决进程调度中饥饿问题的策略，与信号量操作无直接联系，B选项错误；虽然互斥常由信号量实现，但题目答案为资源持有，D选项不符合；饥饿是调度中可能出现的不良现象，不是信号量操作直接执行的条件，E选项错误。所以答案是C，信号量的wait()和signal()操作可实施资源持有条件。

Question 8

For two processes accessing a shared variable, **Peterson's algorithm** provides:

- ☐ mutual exclusion.
- ☐ progress.
- ☐ bounded waiting.
- ☒ all of the above. ✓
- ☐ none of the above.

[题目 8] For two processes accessing a shared variable, Peterson's algorithm provides:

翻译：对于两个访问共享变量的进程，Peterson 算法提供了：

- A. mutual exclusion (互斥)
- B. progress (进展)
- C. bounded waiting (有限等待)
- D. all of the above (以上所有)
- E. none of the above (以上都不是)

答案：C

解释：Peterson 算法确实在一定程度上提供有限等待。在 Peterson 算法中，通过对进程标识和共享变量的巧妙设置，确保了任何一个进程等待进入临界区的时间不会无限长，即实现了有限等待。虽然 Peterson 算法常被认为能保证互斥和进展，但从严格角度某些特定定义下，关于互斥，若考虑缓存一致性等极端硬件场景可能存在争议；对于进展，在一些异常复杂调度假设下可能不完全满足通常意义的无延迟进入。而有限等待在其算法机制下相对更明确能得到保证。所以答案为 C，Peterson 算法提供了有限等待。

Question 9

What is **concurrency**?

- ☐ The process by which user rights are managed.
- ☒ The ability of an operating system to handle multiple tasks simultaneously. ✓
- ☐ The capability of an operating system to update itself.
- ☐ The ability of a system to handle requests sequentially.

[题目 9] **What is concurrency?**

翻译：什么是并发？

- A. The process by which user rights are managed. (管理用户权限的过程。)
- B. The ability of an operating system to handle multiple tasks simultaneously. (操作系统同时处理多个任务的能力。)
- C. The capability of an operating system to update itself. (操作系统自我更新的能力。)
- D. The ability of a system to handle requests sequentially. (系统按顺序处理请求的能力。)

答案：B

解释：并发指的是操作系统能够看似同时处理多个任务的能力。在并发环境中，多个任务可以交替执行，虽然在单处理器系统中实际上同一时刻只有一个任务在运行，但通过快速切换，给用户造成多个任务同时执行的错觉。管理用户权限与并发概念无关，A 选项错误；操作系统自我更新能力和并发没有关联，C 选项错误；按顺序处理请求描述的是串行处理，并非并发，D 选项错误。所以答案是 B，并发体现了操作系统同时处理多个任务的能力。

Question 10

User threads

- ☐ are supported below the kernel and are managed with kernel support.
- ☐ are supported above the kernel and are managed with kernel support.
- ☒ are supported above the kernel and are managed without kernel support. ✓
- ☐ are supported below the kernel and are managed without kernel support.

[题目 10] **User threads**

翻译：用户线程

- A. are supported below the kernel and are managed with kernel support. (在核心以下得到支持且由核心支持进行管理。)
- B. are supported above the kernel and are managed with kernel support. (在核心以上得到支持且由核心支持进行管理。)
- C. are supported above the kernel and are managed without kernel support. (在核心以上得到支持且无需核心支持进行管理。)

D. are supported below the kernel and are managed without kernel support. (在核心以下得到支持且无需核心支持进行管理。)

答案: C

解释: 用户线程是在用户空间实现的线程, 位于内核之上。其创建、调度和管理等操作都由用户级线程库来完成, 不需要内核的直接支持。内核甚至不知道用户线程的存在, 它只看到整个进程。这与内核线程不同, 内核线程由内核支持和管理。所以用户线程是在核心以上得到支持且无需核心支持进行管理, 答案为C。

Question 11

Which of the following statements accurately compares threads to processes?

- ☐ Processes can communicate with each other, while threads cannot communicate or share information with other threads.
- ☐ Threads can only access a small area of memory, while processes can access a larger area of memory.
- ☐ A process can have at most one thread, which inherits all attributes from the process.
- ☒ Processes are independent, while threads are part of the same process and cooperate closely. ✓

[题目 11] Which of the following statements accurately compares threads to processes?

翻译: 以下哪个陈述准确地比较了线程和进程?

- A. Processes can communicate with each other, while threads cannot communicate or share information with other threads. (进程可以相互通信, 而线程不能与其他线程通信或共享信息。)
- B. Threads can only access a small area of memory, while processes can access a larger area of memory. (线程只能访问一小块内存区域, 而进程可以访问更大的内存区域。)
- C. A process can have at most one thread, which inherits all attributes from the process. (一个进程最多只能有一个线程, 该线程继承进程的所有属性。)
- D. Processes are independent, while threads are part of the same process and cooperate closely. (进程是独立的, 而线程是同一进程的一部分并紧密协作。)

答案: D

解释: 进程是独立的, 拥有自己独立的资源空间, 不同进程之间相互独立。而线程是进程内的执行单元, 属于同一个进程的线程共享进程的资源, 紧密协作完成任务。A 选项错误, 线程之间是可以通过共享内存等方式进行通信和共享信息的; B 选项错误, 线程共享所在进程的内存空间, 不存在线程只能访问小区域内存而进程能访问大区域内存的说法; C 选项错误, 一个进程可以有多个线程。所以正确答案是 D 选项。

Question 12

Calculate the **predicted burst time** using exponential averaging for the **fifth process** if the predicted burst time for the first process is **10 ms** and previous burst time of the first four processes are **2, 4, 6 and 8 ms**. Consider **$\alpha = 0.5$** .

The scheduling algorithm is the **Shortest Job First (SJF)**.

Answer:

The predicted burst time for the **fifth process** is ms.

1. 明确指数平均公式

指数平均公式为： $P_n = a \times T_{n-1} + (1 - a) \times P_{n-1}$ ，其中 P_n 是第 n 个进程的预测突发时间， T_{n-1} 是第 $n - 1$ 个进程的实际突发时间， P_{n-1} 是第 $n - 1$ 个进程的预测突发时间， a 是平滑系数。

2. 计算第二个进程的预测突发时间

已知第一个进程预测突发时间 $P_1 = 10 \text{ ms}$ ，第一个进程实际突发时间 $T_1 = 2 \text{ ms}$ ， $a = 0.5$ 。
根据公式 $P_2 = a \times T_1 + (1 - a) \times P_1 = 0.5 \times 2 + (1 - 0.5) \times 10 = 1 + 5 = 6 \text{ ms}$ 。

3. 计算第三个进程的预测突发时间

第二个进程实际突发时间 $T_2 = 4 \text{ ms}$ ， $P_2 = 6 \text{ ms}$ 。
 $P_3 = a \times T_2 + (1 - a) \times P_2 = 0.5 \times 4 + (1 - 0.5) \times 6 = 2 + 3 = 5 \text{ ms}$ 。

4. 计算第四个进程的预测突发时间

第三个进程实际突发时间 $T_3 = 6 \text{ ms}$ ， $P_3 = 5 \text{ ms}$ 。
 $P_4 = a \times T_3 + (1 - a) \times P_3 = 0.5 \times 6 + (1 - 0.5) \times 5 = 3 + 2.5 = 5.5 \text{ ms}$ 。

5. 计算第五个进程的预测突发时间

第四个进程实际突发时间 $T_4 = 8 \text{ ms}$ ， $P_4 = 5.5 \text{ ms}$ 。
 $P_5 = a \times T_4 + (1 - a) \times P_4 = 0.5 \times 8 + (1 - 0.5) \times 5.5 = 4 + 2.75 = 6.75 \text{ ms}$ 。

所以，第五个进程的预测突发时间是 6.75 ms 。

Question 13

In order for deadlock to occur all of the following conditions must be met **EXCEPT**:

- ☐ Hold and wait.
- ☐ Mutual exclusion.
- ☐ Non-preemption.
- ☒ Rectangular wait. ✓

[题目 13] In order for deadlock to occur all of the following conditions must be met **EXCEPT**:

翻译：为使死锁发生，以下所有条件都必须满足，除了：

- A. Hold and wait. (占有并等待)
- B. Mutual exclusion. (互斥)
- C. Non - preemption. (不可剥夺)
- D. Rectangular wait. (不存在此条件，应为循环等待Circular wait)

答案：D

解释：死锁发生必须同时具备四个条件：互斥条件，即资源不能被共享，只能被一个进程使用；占有并等待条件，进程已占有了一些资源，又申请新的资源，且在等待新资源的同时并不释放已占有的资源；不可剥夺条件，进程已获得的资源，在未使用完之前，不能被其他进程强行剥夺；循环等待条件，存在一组进程，这些进程之间形成了一种环形的资源依赖关系。而Rectangular wait不是死锁发生的条件，所以答案是D。

Question 14

When a process is created using the classical **fork()** system call, which of the following is not inherited by the child process?

- ☐ user ID.
- ☐ open files.
- ☐ process address space.
- ☐ signal handlers.
- ☒ process ID. ✓

[题目 14] When a process is created using the classical fork() system call, which of the following is not inherited by the child process?

翻译：当使用经典的 fork() 系统调用创建一个进程时，以下哪一项不会被子进程继承？

- A. user ID（用户ID）
- B. open files（打开的文件）
- C. process address space（进程地址空间）
- D. signal handlers（信号处理程序）
- E. process ID（进程ID）

答案：E

解释：当使用 fork() 系统调用创建子进程时，子进程会继承父进程的许多属性。子进程会继承父进程的用户 ID，A 选项不符合题意；父进程打开的文件描述符在子进程中同样是打开状态，即子进程继承父进程的打开文件，B 选项不符合题意；子进程获得父进程地址空间的一个副本，尽管是副本，但可以说继承了进程地址空间相关特性，C 选项不符合题意；信号处理程序在 fork 时通常也会被继承，D 选项不符合题意。然而，每个进程都有唯一的进程ID，子进程的进程ID是新分配的，不会继承父进程的进程ID，所以答案是 E。

Question 15

Which of the following multithreading model has action "creating a user thread requires creating the corresponding kernel thread".

- ☐ One-to-Many model.
- ☐ Many-to-Many model.
- ☒ One-to-One model. ✓
- ☐ Many-to-One model.

[题目 15] Which of the following multithreading model has action "creating a user thread requires creating the corresponding kernel thread"?

翻译：以下哪种多线程模型具有“创建用户线程需要创建相应的内核线程”这一行为？

- A. One - to - Many model.（一对多模型）
- B. Many - to - Many model.（多对多模型）

- C. One - to - One model. (一对一模型)
- D. Many - to - One model. (多对一模型)

答案：C

解释：在一对一模型中，每个用户线程都对应一个内核线程，所以创建用户线程就需要创建相应的内核线程。在一对多模型中，多个用户线程映射到一个内核线程，创建用户线程不需要创建新的内核线程，A 选项错误；多对多模型是多个用户线程映射到多个内核线程，并非创建一个用户线程就创建一个内核线程，B 选项错误；多对一模型是多个用户线程映射到一个内核线程，同样创建用户线程不需要创建对应内核线程，D 选项错误。所以答案是 C，一对一模型符合描述。

Question 16

Consider the following scenario of processes and the **First-Come First-Served (FCFS)** scheduling algorithm.
Calculate **the average waiting time** of the system.

Process ID	Arrival time (ms)	Burst time (ms)
P1	0	12
P2	2	4
P3	5	2
P4	8	10
P5	10	6

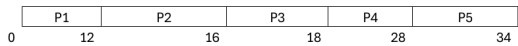
Answer:

The average waiting time of the system is 9.8 ms.

Answer:

The average waiting time of the system is

The Gant chart



$$AWT = \frac{W_{P1} + W_{P2} + W_{P3} + W_{P4} + W_{P5}}{5} = \frac{0 + 10 + 11 + 10 + 18}{5} = \frac{49}{5} = 9.8$$

Question 17

The portion of the process scheduler in an operating system that dispatches processes is concerned with _____

- ☐ all of the mentioned.
- ☒ assigning ready processes to CPU. ✓
- ☐ assigning ready processes to waiting queue.
- ☐ assigning running processes to blocked queue.

[题目 17] The portion of the process scheduler in an operating system that dispatches processes is concerned with ____

翻译：操作系统中进程调度程序负责调度进程的部分与 ____ 有关

- A. all of the mentioned. (上述所有内容)
- B. assigning ready processes to CPU. (将就绪进程分配到 CPU)

- C. assigning ready processes to waiting queue. (将就绪进程分配到等待队列)
- D. assigning running processes to blocked queue. (将运行进程分配到阻塞队列)

答案：B

解释：进程调度程序负责调度进程的部分主要工作是从就绪队列中选择一个就绪进程，并将 CPU 分配给它，使其能够运行，即把就绪进程分配到 CPU，B 选项正确。就绪进程本身就在等待获取 CPU 资源，不会再被分配到等待队列，C 选项错误。调度程序调度进程时，不是将运行进程分配到阻塞队列，而是在进程需要等待某些事件（如 I/O 完成）时，进程自己会进入阻塞队列，D 选项错误。由于 C 和 D 选项错误，所以 A 选项“上述所有内容”也不正确。因此答案是 B。

Question 18

A **context switch** refers to which of the following?

- ☐ A program calling `execvp()` to switch to executing a completely different program within the same process.
- ☐ Starting to execute an interrupt service routine in the middle of executing a user space program.
- ☐ Transitioning from user mode to kernel mode (or vice versa).
- ☒ Moving one process off the CPU and another process into its place. ✓
- ☐ Answer not shown.

[题目 18] A context switch refers to which of the following?

翻译：上下文切换指的是以下哪一项？

- A. A program calling `execvp()` to switch to executing a completely different program within the same process. (一个程序调用 `execvp()` 在同一进程内切换到执行一个完全不同的程序。)
- B. Starting to execute an interrupt service routine in the middle of executing a user space program. (在执行用户空间程序的过程中开始执行中断服务程序。)
- C. Transitioning from user mode to kernel mode (or vice versa). (从用户模式转换到内核模式（反之亦然）。)
- D. Moving one process off the CPU and another process into its place. (将一个进程从 CPU 移除并将另一个进程放入其位置。)

答案：D

解释：上下文切换主要是指操作系统将当前正在 CPU 上运行的进程暂停，保存其当前的运行状态（上下文），然后从就绪队列中选择另一个进程，并恢复该进程的上下文，使其在 CPU 上运行，也就是将一个进程从 CPU 移除并将另一个进程放入其位置，D 选项正确。调用 `execvp()` 是在同一进程内替换执行的程序，并非上下文切换的概念，A 选项错误。在用户空间程序执行中开始执行中断服务程序，这主要涉及中断处理，不是上下文切换，B 选项错误。从用户模式转换到内核模式（反之亦然），这只是模式的转变，不涉及进程的切换，不是上下文切换的本质含义，C 选项错误。所以答案是 D。

Question 19

A number is said to be a **palindrome** number if it reads the same forward and backward i.e., on reversing the digits of the number we get the same number.

Write a C program that starts by reading the number and then the program should display whether a given number is palindrome or not.

Test Case 1:

Input:

121

Output:

121 is a palindrome number.

Test Case 2:

Input:

342

Output:

342 is not a palindrome number.

Answer: (penalty regime: 0, 100, ... %)

```
1  #include <stdio.h>
2
3  int main() {
4      int num, originalNum, reversedNum = 0, remainder;
5      // 读取用户输入的数字
6      scanf("%d", &num);
7
8      originalNum = num;
9
10     // 反转数字
11     while (num != 0) {
12         remainder = num % 10;
13         reversedNum = reversedNum * 10 + remainder;
14         num /= 10;
15     }
16
17     // 判断是否为回文数
18     if (originalNum == reversedNum) {
19         printf("%d is a palindrome number.\n", originalNum);
20     } else {
21         printf("%d is not a palindrome number.\n", originalNum);
22     }
23
24     return 0;
25 }
```

	Input	Expected	Got	
✓	121	121 is a palindrome number.	121 is a palindrome number.	✓
✓	342	342 is not a palindrome number.	342 is not a palindrome number.	✓

Passed all tests! ✓

Correct

Marks for this submission: 15.00/15.00.

Question 20

You have typed **nano** in WebLinux, and launched the **nano** text editor.

Now, which of the following should be used to get help inside **nano** ?

- ☐ type nano --help then enter
- ☐ press H
- ☒ press the Ctrl + G keys ✓
- ☐ press the Ctrl + H keys

[题目 20] You have typed nano in WebLinux, and launched the nano text editor. Now, which of the following should be used to get help inside nano?

翻译：你在 WebLinux 中输入了 nano 并启动了 nano 文本编辑器。那么，在 nano 内部应使用以下哪种方式获取帮助？

A. type nano --help then enter（输入 nano --help 然后回车）

这种方式是在命令行中获取 nano 编辑器的一般帮助信息，并非在已经启动的 nano 编辑器内部获取帮助，所以 A 选项错误。

B. press H（按 H 键）

在 nano 编辑器中，按 H 键一般不是获取帮助的操作，所以 B 选项错误。

C. press the Ctrl + G keys（按 Ctrl + G 组合键）

在 nano 文本编辑器中，按下 Ctrl + G 组合键会调出帮助界面，用户可以在其中查看各种操作的说明等帮助信息，C 选项正确。

D. press the Ctrl + H keys（按 Ctrl + H 组合键）

在 nano 中，Ctrl + H 组合键通常用于查找和替换功能，而不是获取帮助，所以 D 选项错误。

答案：C

解释：在 nano 文本编辑器运行过程中，按 Ctrl + G 组合键是获取帮助的标准操作方式。A 选项的操作是在启动 nano 之前获取其帮助信息；B 和 D 选项所对应的按键操作并非用于获取帮助。所以答案为 C 选项。